

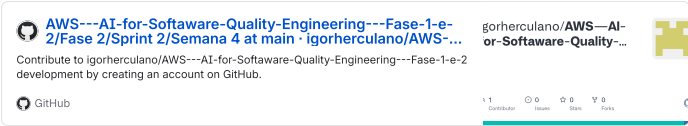
Planejamento ServeRest v2.0

1. Informações Gerais

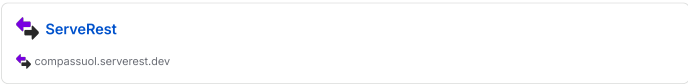
Este documento define o Plano de Testes para a Challenge da semana 4, seguindo uma abordagem simplificada e focada nos requisitos essenciais.

1.1. Identificação do Projeto/Sistema

- **Nome do Projeto/Sistema:** Planejamento de Teste - API ServerRest
- **Data de Criação do Plano:** 02 de fev. de 2026
- **Elaborado por:** Igor Herculano Silva
- **Repositorio:**



- **API testada:**



1.2. Objetivos e Escopo dos Testes

Este plano foca em reconhecer e validar as regras de negocio a partir dos dados documentos no Swagger.

- **Objetivos dos Testes:**
 - Verificar **endpoints**
 - Verificar o **fluxo de valor**
 - Verificar funcionamento dos **tokens**
 - Verificar as regras de negocio aplicadas
 - Verificar conformidade das documentação

1.3. Escopo (Por modulo)

Usuário	Login	Produto	Carrinho	AUTH
Cadastro (POST)	Geração de Token	Cadastro (POST)	Cadastro (POST)	Cadastro (POST)
Busca,listagem (GET)	-	Busca, listagem (GET)	Busca, listagem (GET)	Busca,listagem (GET)
Edição (PUT)	-	Edição (PUT)	Exclusão (DELETE)	Edição (PUT)
Exclusão (DELETE)	-	Exclusão (DELETE)		Exclusão (DELETE)

1.4. Fora do escopo

- Testes de **Carga e Stress**
- **Interface**

2. Abordagem de Teste

A **abordagem de teste será automatizada**, focada na validação de contratos, códigos de status HTTP, regras de negócio e segurança (autenticação e autorização). O desenvolvimento e a execução dos scripts utilizaram o **Robot Framework** em conjunto com a **RequestsLibrary**, tendo a documentação oficial (Swagger) da API ServeRest como base analítica para a validação dos contratos.

2.1. Estratégia de Teste

A estratégia consistiu na criação de uma suíte de testes funcionais estruturada de forma modular, com os Casos de Teste divididos por domínio de negócio (Login, Usuários, Produtos e Carrinhos). A arquitetura do projeto aplicou fortemente o Princípio **DRY** (*Don't Repeat Yourself*), estabelecendo uma separação clara entre os arquivos de Teste e os arquivos de base (Resources).

2.2. Tipos de Teste

- **Testes de Contrato:** Validação da estrutura do JSON de resposta e Tipagem de dados.
- **Testes Funcionais:** Verificação das regras de negócio (CRUD) e Status Codes (200, 201, 400, 401).
- **Testes de Segurança Básica:** Validação de acesso a rotas protegidas sem token ou com token inválido.

3. Ambiente de Teste

3.1. Necessidades de Hardware e Software

Tipo	Especificação
Servidor de Aplicação	Ambiente Externo de Produção
Estações de Teste	Windows 11, Python 3.12.2, Robot Framework 7.4.2
Documentação	Swagger

3.2. Dados de Teste

Os dados do teste serão de uma conta real criada para fins de teste.

4. Critérios de entrada

4.1. Critérios de entrada

O ciclo de testes só poderá iniciar quando:

- A API estiver acessível publicamente na URL `compassuol.serverest.dev/`
- Os casos de testes estiverem finalizados e configurados através da Robot Framework
- O ambiente no Jira estiver organizado para criação de ciclos e bugs.
- A documentação (Swagger) estiver disponível para consulta.
- O documento de Plano de Testes estiver finalizado e entregue na plataforma de ensino (Learning).

4.2. Critérios de saída

- 100% dos Cenários de Teste planejados (Seção 6) forem executados.
- Todos os defeitos encontrados estiverem devidamente reportados no Jira com evidências.
- A taxa de realização dos testes de "Caminho Feliz" (Reconhecimento da API) for 100%.

5. Gerenciamento de Teste

5.1. Gerenciamento de Não Conformidades (Defeitos)

Os defeitos encontrados serão catalogados no **Jira** como *Bugs*, seguindo rigorosamente o template abaixo:

Template de Report de Bug:

6. Cenários de teste funcionais

ID	Pilar (VADER)	Cenário de Teste	Objetivo Técnico	Resultado Esperado
CT-01 - Login	Verbs	Enviar PUT, GET e DELETE na rota /login	Validar se a API bloqueia verbos não suportados.	Status 405 Method Not Allowed.
CT-02 - Login	Data	Tentar realizar login com e-mail inválido e com senha vazia	Validar formato do e-mail e obrigatoriedade de campos de credencial.	Status 400 Bad Request.
CT-03 - Login	Authorization / Errors	Tentar login com senha incorreta e usuário não cadastrado	Garantir bloqueio de credenciais inválidas ou inexistentes no banco.	Status 401 Unauthorized. Msg: "Email e/ou senha inválidos".
CT-04 - Login	Data	Tentar realizar login enviando número no lugar de texto	Validar rejeição de tipos de dados incorretos no payload.	Status 400 Bad Request.

CT-05 - Login	Data	Tentar realizar login com payload completamente vazio	Garantir que a API trate um JSON vazio sem estourar Erro 500.	Status 400 Bad Request.
CT-01 - Carrinho	Data	Tentar cadastrar um segundo carrinho para o mesmo usuário	Validar regra de negócio: limite de 1 carrinho ativo por conta.	Status 400 Bad Request. Msg: "Não é permitido ter mais de 1 carrinho".
CT-02 - Carrinho	Data	Tentar cadastrar carrinho com quantidade maior que o estoque	Verificar se a API valida disponibilidade no estoque de Produtos.	Status 400 Bad Request. Msg: "Produto não possui quantidade suficiente".
CT-03 - Carrinho	Data	Tentar cadastrar carrinho com ID de produto duplicado	Validar exigência de alteração na "quantidade" em vez de duplicar IDs.	Status 400 Bad Request. Msg: "Não é permitido possuir produto duplicado".
CT-04 - Carrinho	Authorization	Tentar cadastrar carrinho sem enviar Token de autorização	Verificar barreira de autenticação na criação de entidades.	Status 401 Unauthorized. Msg: "Token de acesso ausente...".
CT-05 - Carrinho	Verbs	Concluir compra de carrinho válido (DELETE /concluir-compra)	Garantir fluxo de conclusão, baixa de estoque e exclusão do carrinho.	Status 200 OK. Msg: "Registro excluído com sucesso".
CT-01 - Produto	Authorization	Tentar cadastrar produto com usuário comum logado	Validar controle de acesso (RBAC) bloqueando não-administradores.	Status 403 Forbidden. Msg: "Rota exclusiva para administradores".
CT-02 - Produto	Data	Tentar cadastrar produto com nome já existente no banco	Verificar bloqueio de duplicidade de registros.	Status 400 Bad Request. Msg: "Já existe produto com esse nome".
CT-03 - Produto	Data	Tentar cadastrar produto com preço negativo	Validar sanitização de valores financeiros inválidos.	Status 400 Bad Request. Msg: "preco deve ser um número positivo".
CT-04 - Produto	Authorization	Tentar cadastrar produto sem enviar Token no header	Garantir que rotas de manipulação exijam token Bearer.	Status 401 Unauthorized. Msg: "Token de acesso ausente...".
CT-05 - Produto	Verbs	Executar fluxo completo de CRUD (POST, GET, PUT, DELETE)	Validar ciclo de vida funcional e interligação de métodos HTTP.	Status 201 e 200 OK.
CT-01 - Usuário	Data	Tentar cadastrar usuário com email já em uso	Garantir bloqueio de duplicidade de chaves únicas no banco.	Status 400 Bad Request. Msg: "Este email já está sendo usado".
CT-02 - Usuário	Data	Tentar cadastrar usuário com formato de email sem arroba/domínio	Verificar validação de contrato (Joi) para formato de e-mail.	Status 400 Bad Request. Msg: "email deve ser um email válido".
CT-03 - Usuário	Data	Tentar cadastrar usuário com campo de senha vazio	Validar obrigatoriedade e tratamento de campos em branco.	Status 400 Bad Request. Msg: "password não pode ficar em branco".

7. Riscos

Risco (Falha Provável)	Probabilidade	Impacto	Exemplo possíveis
Falso Positivo em Autenticação	Média	Crítico	A API permitir acesso a rotas de Admin usando token de usuário comum.
Inconsistência de Dados (Lixo no Banco)	Média	Médio	A API aceitar cadastro de produtos com preço negativo ou nomes vazios.

Persistência Volátil	Alta	Baixo	O servidor reiniciar e apagar os usuários criados no teste anterior.
----------------------	------	-------	--

8. Mapeamento de Endpoints

Módulo	Método	Endpoint	Objetivo Principal
Login	POST	/login	Autenticar usuário e gerar Token.
Usuários	GET	/usuarios	Listar todos os cadastros.
	POST	/usuarios	Cadastrar novo usuário.
	DELETE	/usuarios/{_id}	Excluir usuário.
Produtos	GET	/produtos	Listar produtos disponíveis.
	POST	/produtos	Cadastrar novo produto (Requer Token).
Carrinhos	POST	/carrinhos	Criar carrinho com produtos.
	DELETE	/carrinhos/concluir-compra	Finalizar compra e limpar carrinho.
	DELETE	/carrinhos/cancelar-compra	Cancelar compra e limpar carrinho.

9. Exemplos de requisição

1. Login

Endpoint: /login

POST (Autenticação)

JSON

```
1 {
2   "email": "fulano.qa@teste.com.br",
3   "password": "teste"
4 }
```

2. Usuários

Endpoint: /usuarios

POST (Cadastro de Usuário)

JSON

```
1 {
2   "nome": "Igor QA Teste",
3   "email": "igor.qa.teste1@teste.com.br",
4   "password": "teste",
5   "administrador": "true"
6 }
```

PUT (Edição de Usuário) URL: /usuarios/{_id}

JSON

```
1 {
2   "nome": "Igor QA Editado",
3   "email": "igor.qa.teste1@teste.com.br",
4   "password": "nova_senha_123",
5   "administrador": "true"
6 }
```

3. Produtos

Endpoint: /produtos *Requisito: Header* Authorization: Bearer <seu_token>

POST (Cadastro de Produto)

JSON

```
1 {
2   "nome": "Notebook Gamer",
3   "preco": 12000,
4   "descricao": "Notebook para jogos de alta performance",
5   "quantidade": 10
6 }
```

PUT (Edição de Produto) URL: /produtos/{_id}

JSON

```
1 {
2   "nome": "Notebook Gamer
3   V2",
4   "preco": 11500,
5   "descricao": "Versão atualizada com mais RAM",
6   "quantidade": 15
7 }
```

4. Carrinhos

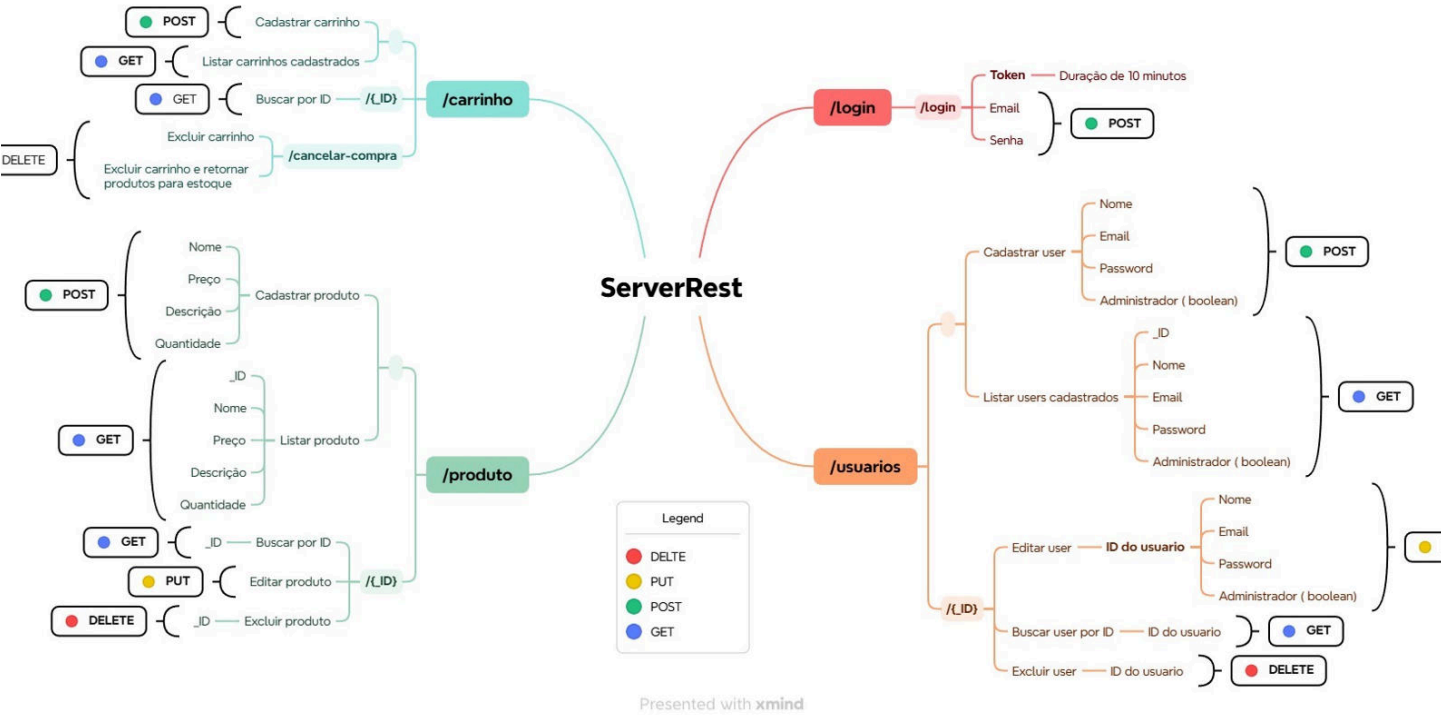
Endpoint: /carrinhos

POST (Criar Carrinho)

JSON

```
1 {
2   "produtos": [
3     {
4       "idProduto": "BeeJh5Lz3k6kSIzA",
5       "quantidade": 1
6     },
7     {
8       "idProduto": "K6leHdftCe0jj8BJ",
9       "quantidade": 2
10    }
11  ]
12 }
```

10. Mapa Mental



11. Bugs Encontrados

Type	Key	Resumo	Responsável	Prioridade	Status	Atualizado(a)
🚩	SCRUM-18	Falta de Validação em Campos Obrigatórios (Nome e Password) Vazios	👤 igor H.	🔴 High	EM ANDAMENTO	20 de mar. de 2026, 12:11
🚩	SCRUM-16	Bug encontrado na documentação ou no message de resposta na parte de Excluir Car...	👤 igor H.	🔵 Low	EM ANDAMENTO	19 de mar. de 2026, 15:25

🔄 Synced just now · 2 items

12. Registro de execuções

📄 TestingServerRest Log.pdf

13. Passo a passo de execução

Opção 1: Execução Visual e Individual

Recomendo a instalação da extensão RobotCode no VS Code.

Com ela instalada, botões de "Play" (🎮) aparecerão automaticamente ao lado de cada Caso de Teste e no topo de cada arquivo .robot. Isso permite que você rode:

- Apenas um Caso de Teste isolado (ideal para debugar).
- Um arquivo/suite inteira por vez (ex: rodar apenas os testes de Carrinho).

Opção 2: Execução via CMD (Terminal)

Para executar os testes diretamente pelo terminal, navegue até a pasta do projeto TestingServerRest e execute o comando abaixo:

```
robot ServerRest_testes_1HP.robot ServerRest_testes_CARRINHO.robot ServerRest_testes_LOGIN.robot
ServerRest_testes_PRODUTOS.robot ServerRest_testes_USUARIOS.robot
```

Ou, para rodar todos os testes ao mesmo tempo:

```
robot .
```

O caminho até a pasta de testes é:

